

刀与算法：渗透工具智能化与 *AI Native*渗透测试之争

演讲人：V1ll4n

Yak Project 负责人 / 万径安全 CTO



About me

V1ll4n (姬锦坤)

万径安全 CTO

Yak Project 项目负责人

2025年度通信学会科技进步一等奖

2023年度信息通信领域十大科技进展

《CDSL-YAK网络安全领域编程语言》

Product: Yakit / Irify / Memfit AI



Content 目录

PART 01 背景: AI 与工具协同进化

PART 02 两种演化路径

PART 03 真实战场: 碎片场景

PART 04 基础设施仍然重要

01

背景：AI与工具协同进化

从经济学的价值讨论出发

人工智能到底对“渗透测试”做了什么？

工具在其中扮演了怎么样的位置？



核心：AI 正在压缩“社会必要劳动时间”

知识结构化 + 流程标准化 ⇒ 低水平重复劳动可替代性上升 ⇒ 市场稀缺性下降 ⇒ 价值被压缩

AI 正在压缩渗透测试基础劳动价值

知识结构化 + 流程标准化

⇒ 枚举 / 识别 / 验证 / 整理 环节被 AI 加速接管

⇒ 依赖工具熟练度和流程执行的能力不再稀缺

⇒ 渗透测试中基础劳动价值被压缩

HACK.SKILLS - Hacker Arsenal for Agents



Master Entry → Category Entries → Deep Topic Skills

One master entry, six category entries, and 100 deep topic skills across 14 security domains.

An Agent Skills knowledge base covering web security, API security, authentication & authorization, OS privilege escalation (Linux/Windows/macOS), Active Directory attacks, mobile security, binary exploitation (Pwn), reverse engineering, cryptography attacks, blockchain & smart contract security, AI/ML & LLM security, network protocols & pivoting, and digital forensics — built for bug bounty, penetration testing, CTF competitions, and authorized security research.



AI 压缩了基础劳动的价值;
但是工具正在成为 AI 渗透测试落地的“手”和“脚”

真正的变化不是 谁 代替 谁, 而是

AI 与工具开始协同进化

借助《三体》概念
AI 与工具协同也有类似的路线

两大阵营: 拯救派 / 降临派

拯救派: 关注传统工具如何改造,
更好地被 AI 使用, 服务于人

降临派: AI 终将接管一切, 创
造新的秩序, 主张使用AI编排一切

Don't Panic!
尤其是关注我们的工具/产品研发团队

虽然 AI 现如今屠榜各种热点
但是 Yak Project 产品使用量仍然可以保持
在公域人均3.2小时以上(截止四月中旬)

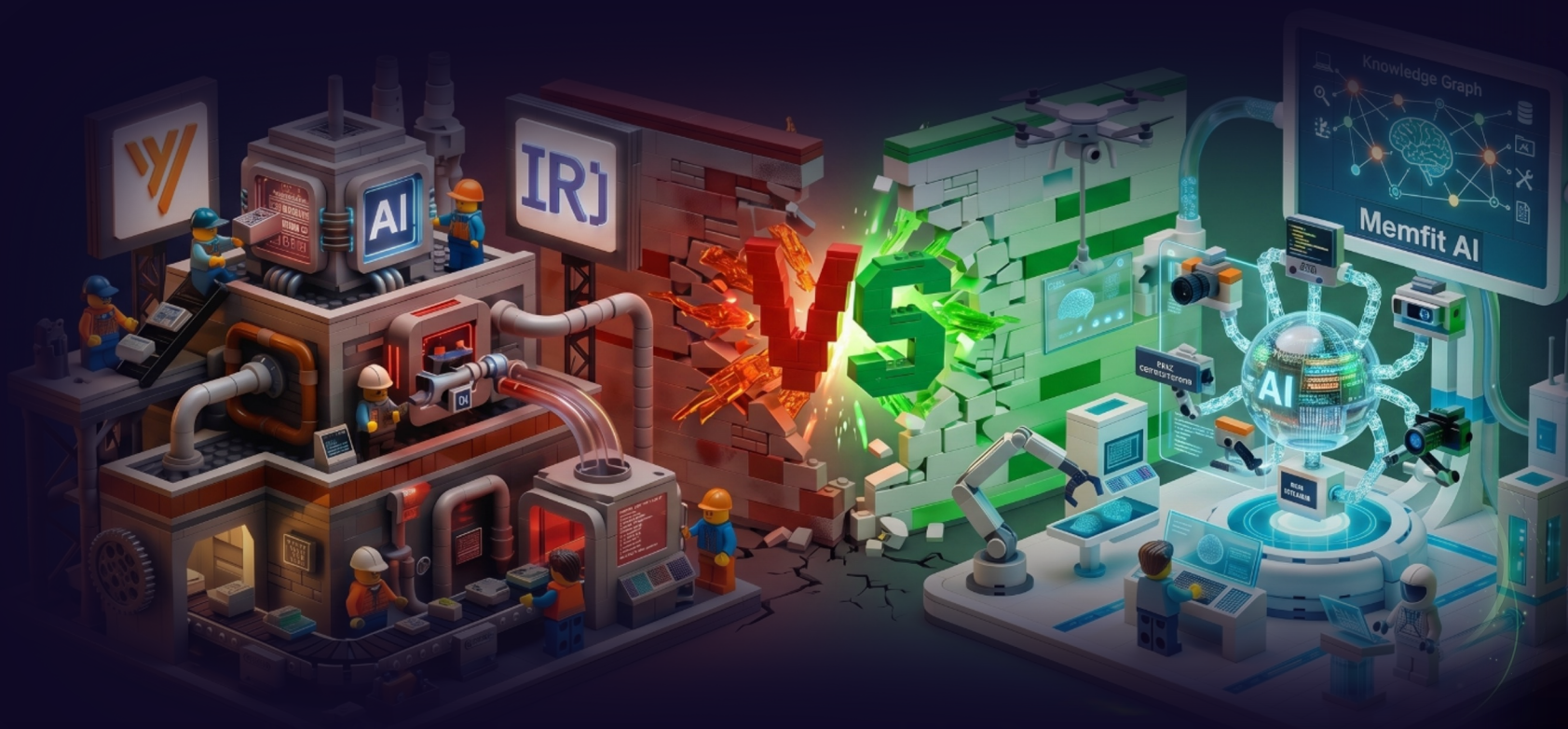
至少目前短平快的工具仍然是工作中的核心



02

两种演化路径

Tool First 还是 AI First?



两种不同的演化路径

维度	拯救派( )	降临派()
理念	Tool First 以工具解决特定场景问题为主, AI 强化工具能力	AI First 围绕纯 AI 重新定义 workflow SOP 和入口
优势	继承生态, 验证路径短 渐进式交付, 实际效果好评估	摆脱旧系统的结构性上限 提前避坑, 上限高
劣势	新的场景无法快速支持 旧的代码架构有 历史包袱	难以覆盖 已经被广泛验证的用户习惯和使用场景
场景	用户规模大 / 存量功能多 兼容旧功能 / 尊重用户习惯	探索新形态 / 重构老产品 希望构建原生新入口

那么, 我们如何付诸行动, 让 Yak Project 产品线更好适应 AI 时代呢?

演化路径：拯救派 [加速迭代] *Tool First, AI Enhance!*

AI 加速工具产研迭代

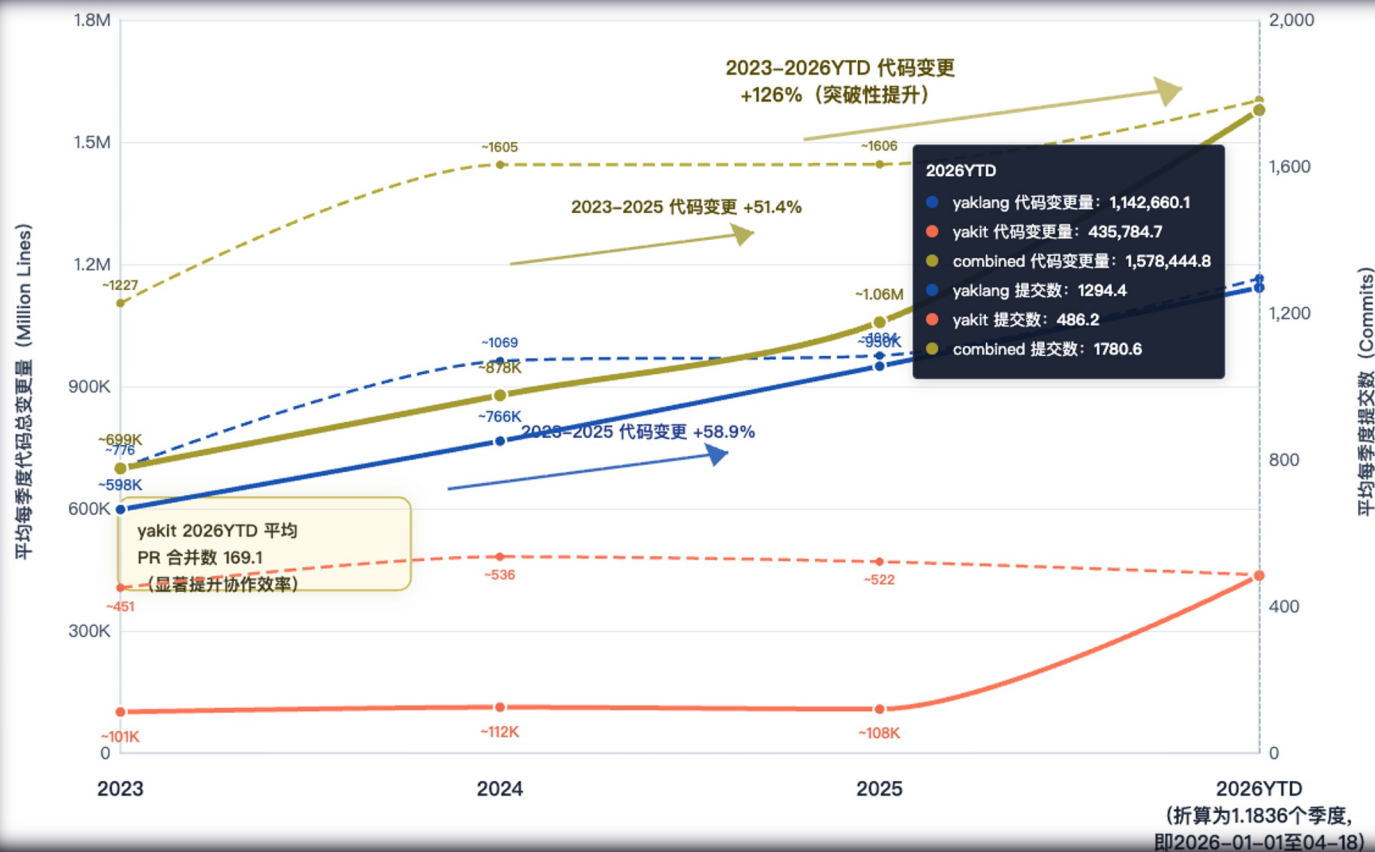
团队没有大规模扩充
季度代码变更总量提升

58.9%

2023 - 2025

126%

2023 - 2026 (截止第一季度)



演化路径：拯救派 [产品级强化]

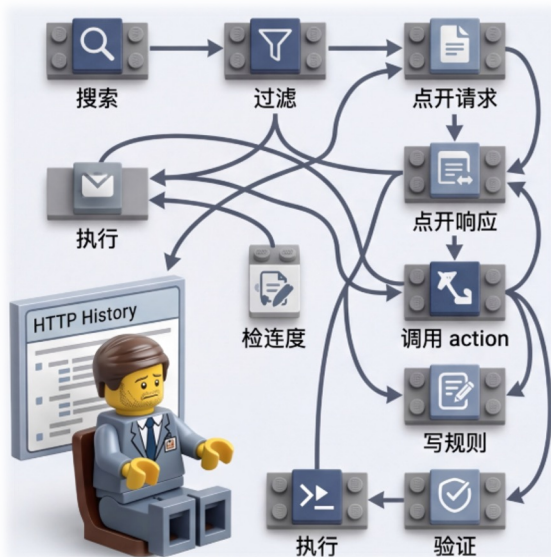
Tool First, AI Enhance!

[案例1]：使用AI优化 HTTP 流量分析

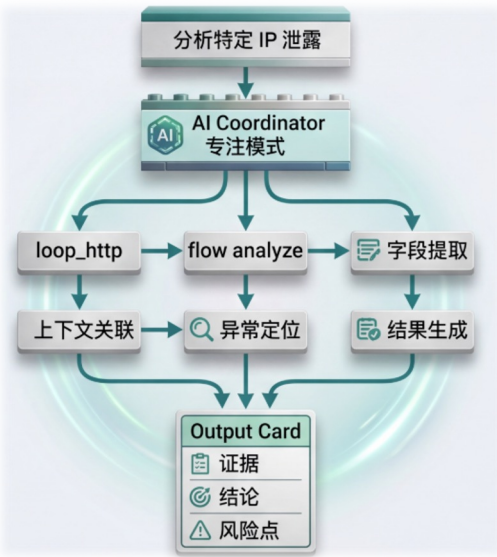
过去：用户必须自主完成工具内的一系列操作：

搜索 -> 过滤 -> 点击 -> 对比 -> 提取 -> 验证

现在：用户表达目标 -> AI执行 -> 用户检查



人工编排，繁琐割裂



AI 编排，专注目标

AI

分析当前流量中有信息泄露的地方吗？

flow-findings memfit-standard-free 2026-04-18 09:47:20

信息泄漏分析启动

- 目标：扫描当前捕获的 HTTP 流量，识别潜在的信息泄漏点。
- 策略：
 - 搜索响应体中包含敏感关键词（如 password, secret, key, token, id, card, phone）的流量。
 - 搜索状态码为 200 但包含错误堆栈或调试信息的流量。
 - 检查响应头中是否泄露服务器版本或内部路径。
- 当前状态：正在执行第一轮广泛搜索，定位包含常见敏感词的响应。

[参考资料]

思考 扩大搜索范围，检查响应头和Cookie中的敏感信息泄露。

flow-findings memfit-standard-free 2026-04-18 09:47:33

初步分析结果

- 状态：第一轮基于常见敏感词（password, secret等）在响应体中
- 发现：在响应体中，而非明文。将重点放在响应体中，如JSON、XML、HTML、Server版本、内部调试头）以及JSON结构中的敏感键名。
- 风险点：重点关注 Set-Cookie 是否包含敏感Session

自定义AI全局指令

请告诉我，你想做什么...(shift + enter 换行)

MITM 插件 爬网站树

序号	方法	URL	状态码
1716	GET		
1715	GET		
1714	GET		
1713	GET		
1712	GET		
1711	GET		
1710	GET		
1709	GET		
1708	GET		
1707	GET		
1706	GET		
1705	GET		
1704	GET		
1703	GET		
1702	GET		
1701	GET		
1700	GET		
1699	GET		
1698	GET		
1697	POST		
1696	POST		
1695	GET		200

严重信息泄露：加密密钥明文返回

- 流量 ID: #1645
- URL:
- 问题描述：服务端在响应体中直接返回了用于加密的 AES Key 和 IV。
- 证据：
 - 请求体包含客户端提供的 key/iv: {"key": ,...}
 - 响应体包含服务端生成的 key/iv: "1c" ,...7b"
- 安全风险：
 - 加密失效：如果密钥通过明文通道传输，中间人攻击者可直接获取密钥解密后续通信或伪造数据。
 - 算法误用：路径暗示使用 AES-ECB 模式，该模式不具备语义安全性，相同明文块产生相同密文块，易受模式分析攻击。
 - 密钥管理缺失：服务端动态生成密钥并返回给客户端，通常意味着密钥未妥善存储或验证机制缺失。

潜在敏感端点列表

- JWT 签名或密钥泄

自定义AI全局指令

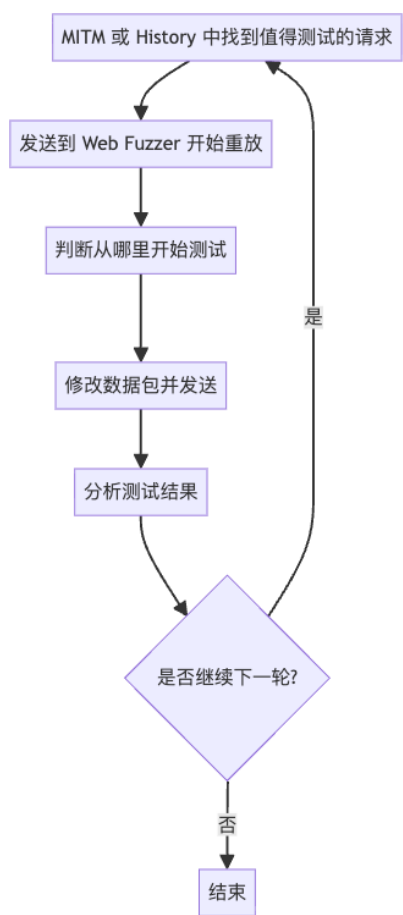
请告诉我，你想做什么...(shift + enter 换行)

HTTP History 中使用 AI 分析流量

演化路径：拯救派 [产品级强化]

Tool First, AI Enhance!

[案例2]：使用AI来操作 Web Fuzzer [发包器]



http://127.0.0.1:8787/xss/js/in-str-temp?name=admin

思考 用户输入为完整URL格式，未提供原始HTTP请求报文，因此提取URL并默认方法为GET

HTTP 数据包

Request

```
1 GET /xss/js/in-str-temp?name=admin HTTP/1.1
2 Host: 127.0.0.1:8787
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/83.0.4103.116 Safari/537.36
4
5
```

18 秒完成测试
发送六个数据包确定结果

思考 开始执行 fuzz_get_params，HTTPFlow 会落库并保留完整请求/响应。

HTTP 数据包 1:8787/xss/js/in-str-temp?name=%3Cimg+src%3Dx+onerror%3Dalert%281%29%3E STATUS: 200 BODY: [(909) bytes]

思考 fuzz_get_params 测试结果已准备完成，开始验证是否达到当前安全测试目标。

核实结果 目标达成：Fuzz测试已完成，发现JS模板字符串内存在XSS/SSTI注入点（如`\${7\*7}`被解析），证据确凿。主体：接口 GET /xss/js/in-str-temp?name=`#xss\_ssti\_in`

思考 fuzz_get_params 目标验证完成：已达到当前安全测试目标。

AI 响应 memfit-qwen3.6-plus-no-thinking-free 2026-04-19 10:24:15

HTTP Fuzz Test 阶段总结

用户目标

http://127.0.0.1:8787/xss/js/in-str-temp?name=admin

当前有效请求

URL: http://127.0.0.1:8787/xss/js/in-str-temp?name=admin BODY: [(0) bytes]

已执行动作

1. 执行了 fuzz_get_params。参数：param_name=name; param_values=[alert(1) ">alert(1) ">alert(1) `\${7\*7}` `\${7\*7}`]; raw_mode=false; reason=URL路径 /xss/js/in-str-temp 暗示该端点可能涉及 JavaScript 模板字符串或动态内容生成。参数 name=admin 是主要的输入点。首先测试反射型 XSS（通过 script 标签和事件处理器）以及服务端模板注入（SSTI，通过数学表达式），以验证输入是否被直接回显或在服务端被执行。响应：共执行 6 次测试，保存 6 条 HTTPFlow。代表性响应状态：200 验证：已达到当前目标：目标达成：Fuzz测试已完成，发现JS模板字符串内存在XSS/SSTI注入点（如`\${7\*7}`被解析），证据确凿。代表性 HTTPFlow: e46252c0-9bc9-484e-9a67-dc923f66b325

当前发现

=== Verification === Satisfied: true Reasoning: 目标达成：Fuzz测试已完成，发现JS模板字符串内存在XSS/SSTI注入点（如`\${7\*7}`被解析），证据确凿。Representative HTTPFlow: e46252c0-9bc9-484e-9a67-dc923f66b325

代表性样本

HTTPFlow: e46252c0-9bc9-484e-9a67-dc923f66b325

退出原因

当前阶段已结束，系统按已有状态生成了这份总结。

给我一些 Payload

思考 Restored the original HTTP packet and latest vulnerability analysis from the current session.

AI 响应 memfit-qwen3.6-plus-no-thinking-free 2026-04-19 10:24:45

基于前一轮 Fuzz 结果，目标端点 /xss/js/in-str-temp 将用户输入直接嵌入 JavaScript 模板字符串（Backticks）中。已确认 `\${7\*7}` 被解析，存在客户端模板注入风险。以下是针对该上下文的进阶 Payload：

[参考资料]

思考 提供JS模板注入Payload

AI 响应 memfit-qwen3.6-plus-no-thinking-free 2026-04-19 10:24:47

JS 模板字符串注入 Payload 集合

漏洞背景

响应体显示输入点位于：

```
javascript
1 console.info("Hello" + `${PAYLOAD}`);
2
```

输入被包裹在反引号 `` 中，且处于 JavaScript 执行上下文。

推荐 Payload

1. 基础表达式执行（验证解析）

- `\${alert(1)}`
- `\${confirm(1)}`
- `\${prompt(1)}`
- `\${7\*7}`（已验证有效）

2. 逃逸与代码注入（突破模板限制）

如果目标是执行任意 JS 代码而非仅表达式：

```
`;alert(1);//
// 原理：闭合模板字符串，结束语句，执行 alert，注释掉剩余代码
```


演化路径：拯救派 [产品级强化]

Tool First. AI Enhance!
能力平台总览架构图

[案例3]：使用 AI 来调度现存 Yakit 或其他能力插件

检查一下这个目标的fastjson漏洞

http://127.0.0.1:8787/fastjson/json-in-query?auth=%7B%22user%22%3A%22admin%22%2C%22password%22%3A%22password%22%7D

意图识别 检测目标是否存在 Fastjson 反序列化漏洞

思考 申请 Fastjson 漏洞检测工具

加载工具 loading tool: Fastjson 综合检测...done! Fastjson 综合检测(Core Tool / 核心工具 / High Priority) is prepared

生成工具参数 通过60s无输出判定超时退出; 成功标准为收到漏洞检测结果报告(存在/不存在/误报排除) "[done] 耗时(Cost): 4.02s [参考资料]

思考 call_expectations: 预估时间、超时处理等 3. 可能通过https://github.com/alibaba/fastjson2/issues/1044了解该漏洞原理及修复方案, 不需要AI/AOC包裹

工具审阅 立即执行 qwen3.5-35b-a3b-free 2026-04-23 16:00:00 目标 fastjson 框架可能存在 RCE 漏洞 (Delay Check)

工具调用 qwen3.5-35b-a3b-free 2026-04-23 16:00:00 漏洞详情 数据包

Fastjson 综合检测 成功

执行成功

[info] sending HTTP request to: http://127.0.0.1:8787/fastjson/json-in-query?auth=%7B%22user%22%3A%22admin%22%2C%22password%22%3A%22password%22%7D

[info] HTTP response: status=200, body={}

[info] analyzing HTTP flow for vulnerability

[info] 无法判断目标 json 框架类型

[info] 目标json框架版本范围是 [0.0.0 - 99.0.0]

解决方案

1. 升级Fastjson版本: 如果你的项目中使用了Fastjson, 请确保使用的是1.2.80版本或以上。Fastjson 1.2.80版本修复了该漏洞。

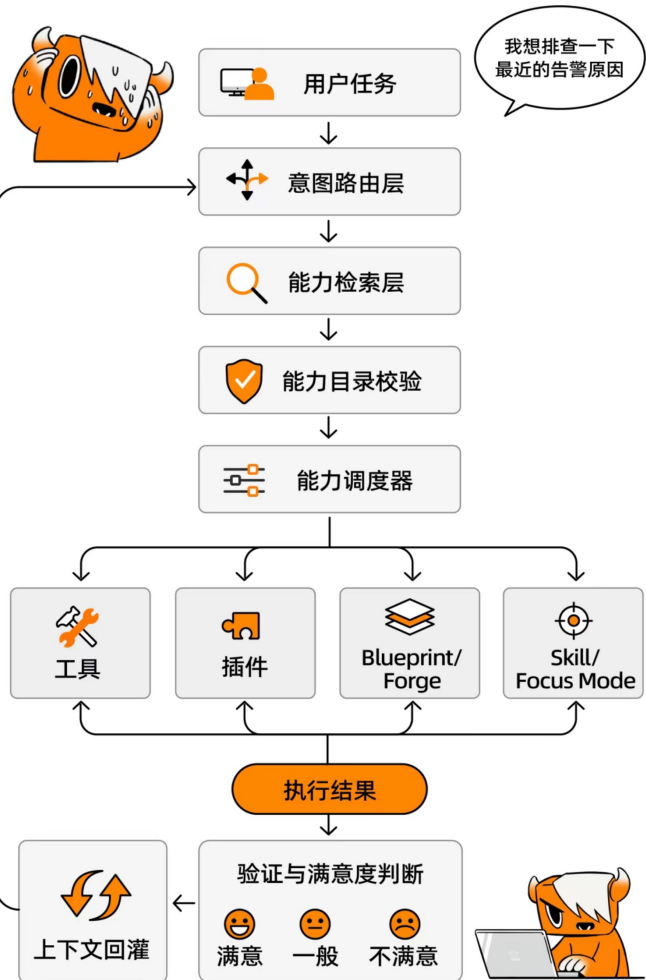
2. 避免使用AutoType特性: Fastjson的AutoType功能是导致反序列化漏洞的一个重要因素。该功能允许将JSON字符串解析为任意Java类。建议禁用该功能, 可以通过设置ParserConfig的autoTypeSupport属性为false来关闭该功能。这样做会限制某些复杂类型的反序列化。

3. 审查输入数据: 在处理JSON字符串时, 对于从不可信来源(如网络、用户输入等)获取的数据, 应进行严格的输入验证和过滤。

进度检查 预估时间: 约 25 秒

思考 'human_readable_thought': '申请 Fastjson 漏洞检测工具'

{ 'human_readable_thought': '申请 Fastjson 漏洞检测工具' }



演化路径：拯救派 [函数级AI强化] *Tool First, AI Enhance!*

函数级 AI 强化工具：验证码识别(解决方案与提升)

维度	传统OCR/平台/手工	AI 多模态
部署成本	OCR 服务 / 逻辑绕过 / 接码系统	复用通用 AI 标准库 + Token 费用
接入复杂度	接口不统一 / 外挂服务	通用 AI 接口，无需定制
识别能力	不同验证码的 OCR / 绕过都不同，不能量化	某 35B-A3B 模型单次识别成功率 57% [4位]，最多6次重试99%
渗透测试影响	容易卡点 无逻辑漏洞基本可放弃	验证码难度不大可规模化爆破

表格中提到的模型，1000次大约5-8毛人民币

演化路径：拯救派 [函数级AI强化]

Tool First, AI Enhance!

函数级 AI 强化工具：验证码识别(解决方案与提升)

```
2026-04-20 08:45:55 [yakit-code-2638721159:48] {"action":"stop","extra":{"ai-model":"memfit-vision-free"}, "streamId":"","streamType":"ai","type":"stream"}
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] trying password: 0014, captcha: 6T4d
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] {"action":"start","extra":{"ai-model":"memfit-vision-free"}, "streamId":"","streamType":"ai","type":"stream"}
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] {"action":"data","data":{"code":"ZJT2"},"extra":{"ai-model":"memfit-vision-free"}, "streamId":"","streamType":"ai","type":"stream"}
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] {"action":"data","data":{"code":"955G"},"extra":{"ai-model":"memfit-vision-free"}, "streamId":"","streamType":"ai","type":"stream"}
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] {"action":"stop","extra":{"ai-model":"memfit-vision-free"}, "streamId":"","streamType":"ai","type":"stream"}
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] {"action":"stop","extra":{"ai-model":"memfit-vision-free"}, "streamId":"","streamType":"ai","type":"stream"}
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] trying password: 0018, captcha: ZJT2
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] trying password: 0015, captcha: 955G
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] {"action":"start","extra":{"ai-model":"memfit-vision-free"}, "streamId":"","streamType":"ai","type":"stream"}
[INFO] 2026-04-20 08:45:55 [yakit-code-2638721159:48] {"action":"start","extra":{"ai-model":"memfit-vision-free"}, "streamId":"","streamType":"ai","type":"stream"}
```

观察实际运行效果

在 Yakit 可配合热加载编写 AI 识别函数
可以直接看结果!

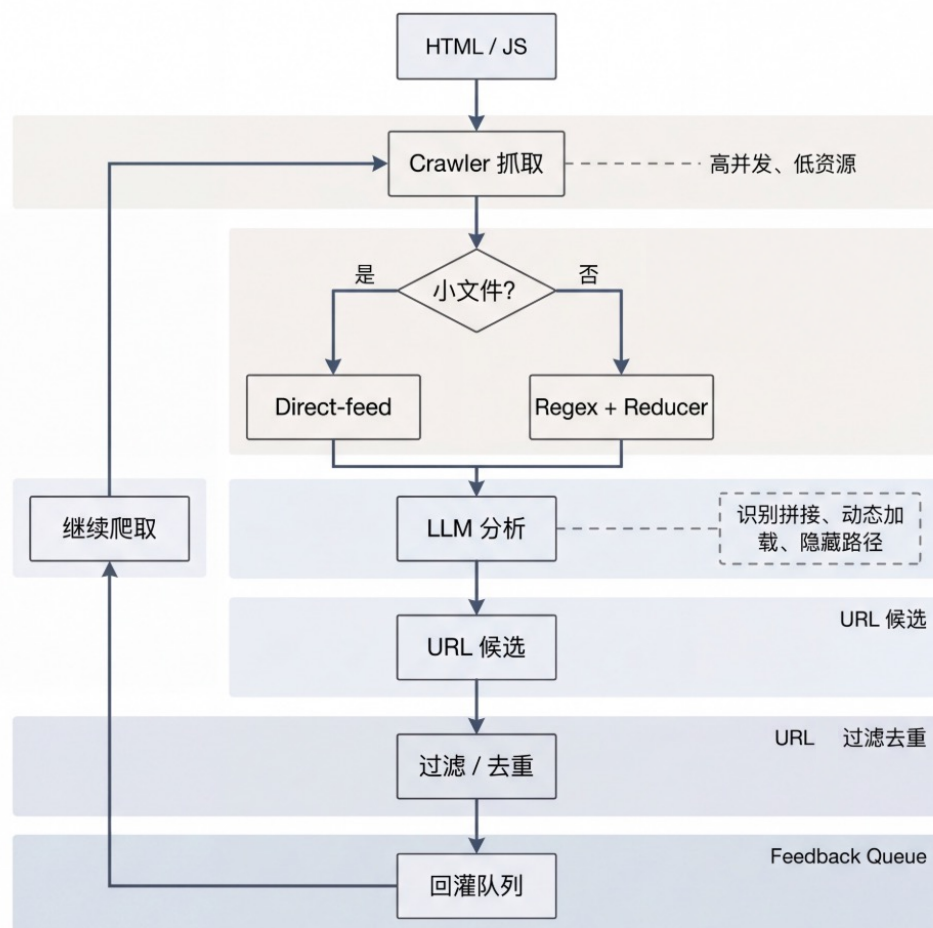
请求	Method	状态	响应大小	延迟(ms)	Payloads	操作
1	POST	500	33	1	0004	🔍 🔄
2	POST	500	33	0	0008	🔍 🔄
3	POST	500	33	0	0016	🔍 🔄
4	POST	500	33	1	0019	🔍 🔄
5	POST	500	33	0	0007	🔍 🔄
6	POST	500	33	0	0013	🔍 🔄
7	POST	500	33	0	0017	🔍 🔄

快速预览	请求	响应
1	HTTP/1.1 500 Internal Server Error	远端地址: 127.0.0.1:8787; 响应时间: 1ms; 总耗时: 1ms; URL:http://127.0.0.1:8787/verifica...
2	Date: Mon, 20 Apr 2026 08:44:11 GMT	
3	Content-Type: text/plain; charset=utf-8	
4	Content-Length: 33	
5		
6		{"code":500,"msg":"密码错误"}

演化路径：拯救派 [函数级AI强化]

Tool First, AI Enhance!

函数级 AI 强化工具：函数级 AI 强化工具：渗透测试用途的爬虫 URL 发现 [SPA 时代资产覆盖率提升]



不执行 JS
不执行 JS
不执行 JS

AI 理解 JS?

1. 20M + 的 JS 可以提取敏感片段
2. 20M 压缩至 6M 左右
3. 使用 AI Reducer 技术规约与提取
4. AI Reducer 切片后并发提取

演化路径：拯救派 [函数级AI强化] Tool First, AI Enhance!

函数级 AI 强化工具：函数级 AI 强化工具：渗透测试用途的爬虫 URL 发现 [SPA 时代资产覆盖率提升]

维度	传统非浏览器普通爬虫	浏览器爬虫(crawlerx / rad)	爬虫基础设施+AI 分析 JS
部署/实现成本	纯编程，资源开销最低	依赖浏览器 [或 Webkit 环境] 一个 Tab 200M内存，需系统图形栈	非浏览器爬虫 + AI Tokens 费用
优势	快速，稳定	交互式浏览器事件处理	稳，准，狠 [速度可控/精度极高]
识别能力	几乎无法支持复杂JS 靠正则凑合 [哑火]	强依赖页面交互，易遗漏不可达资产 受渲染影响导致不稳定，五	浏览器触发不了的URL也可以找到 完全不需要JS执行和HTML渲染
渗透测试影响	SPA 与现代前端应用 无法使用	速度慢，不稳定，内存开销大	时间+AI Tokens换广度 效果稳定，模型越强效果越好

Memfit: qwen3.6-*-free, aiJSConcurrency=3, 单次[20M] Page B 约 60 次 AI 调用

演化路径：降临派 *(Memfit AI)* *Tool First, AI Enhance!*

Memfit 是什么？

Memfit AI 是一个具备战略战术执行，工具技能，知识系统，记忆系统的**智能代理平台**

由 *Yak Project* 生态驱动

开源

到4月20日〔一个月内〕面向用户提供了

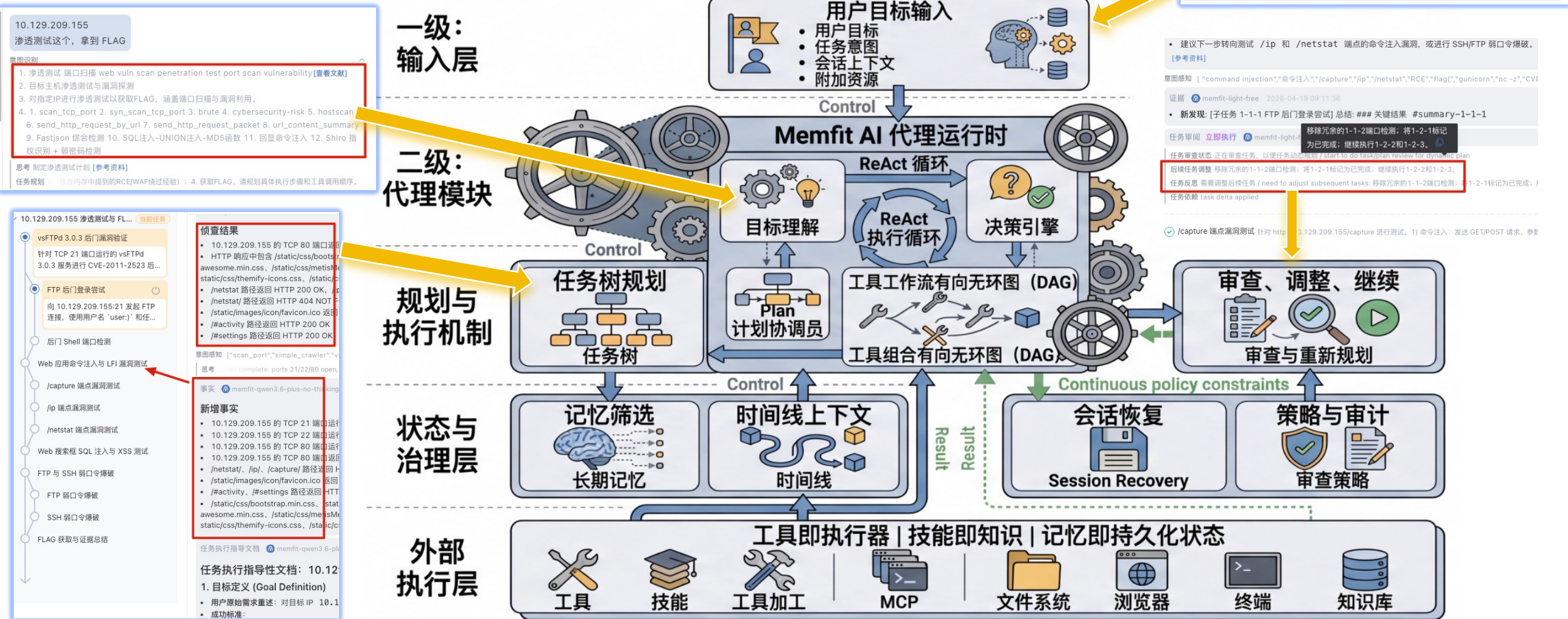
7,640,000次 AI 交互

超300亿 Tokens



维度	Memfit AI 第一代 全能型智能体企业级解决方案
核心架构	递归式双引擎：ReAct+Plan-Execute 深度融合，递归耦合执行
任务规划	AI生成战略规则 → 用户审核 → 任务树分解 → 并行/串行执行
任务分解	智能递归分解，层层递进，子任务独立可控
技能系统	丰富的预置技能库 + 自定义技能扩展
记忆机制	智能记忆系统 + RAG知识增强 + 长期记忆
执行模式	多种专注模式(深度研究、快速执行、创意探索等)
可控性	用户审核机制，关键节点人工介入
透明性	完整的执行链路追溯，细定性日志

演化路径：降临派 *Tool First, AI Enhance!*



演化路径： 降临派 *Tool First, AI Enhance!*

可执行

AI 不只是单轮问答，而且可以持续推进到任务完成

可观测

任务树，时间线，工具调用，系统状态，上下文成分都能透明呈现

可控制

审查策略，人工介入，任务跳过，动态调整，自我反思，自我治理能力

可恢复

会话状态和任务进度可以被持久化，支持跨轮次跨时段运行

可累计

记忆，知识系统，技能和执行结果会沉淀为后续任务的长期资产

核实验结果 目标达成：已获取磁盘使用率、敏感文件

证据 - **New finding**: 主体: macOS文件系统 #fs_ch

待办 memfit-qwen3.6-plus-no-thinking-free

• 使用限制深度或排除大目录的du命令

• (skipped) 使用 timeout 命令限制 du

证据 memfit-qwen3.6-plus-no-thinking-free

• 新发现: 主体: macOS文件系统 #fs_ch

• 新发现: [子任务 1-5 文件系统检查] 核实:

任务验证防止漂移



任务树 / 时间线 / 系统指标 / 上下文成分



人工介入 / 任务控制 / HITL



自动构建记忆系统 / 蒸馏知识库

演化路径并非对立！ 适时适人选择

拯救派

从确定性场景出发，直接解决
真实生产的问题

降临派

从新架构，新能力出发
探索未来入口

Yak Project 的选择是？

对于 Yak Project 来说
我们接受了大量用户托举，大量用户场景反馈

YAKIT / IRIFY  

会使用 AI 来直接优化生产场景

Memfit AI 

来带给用户新的可能

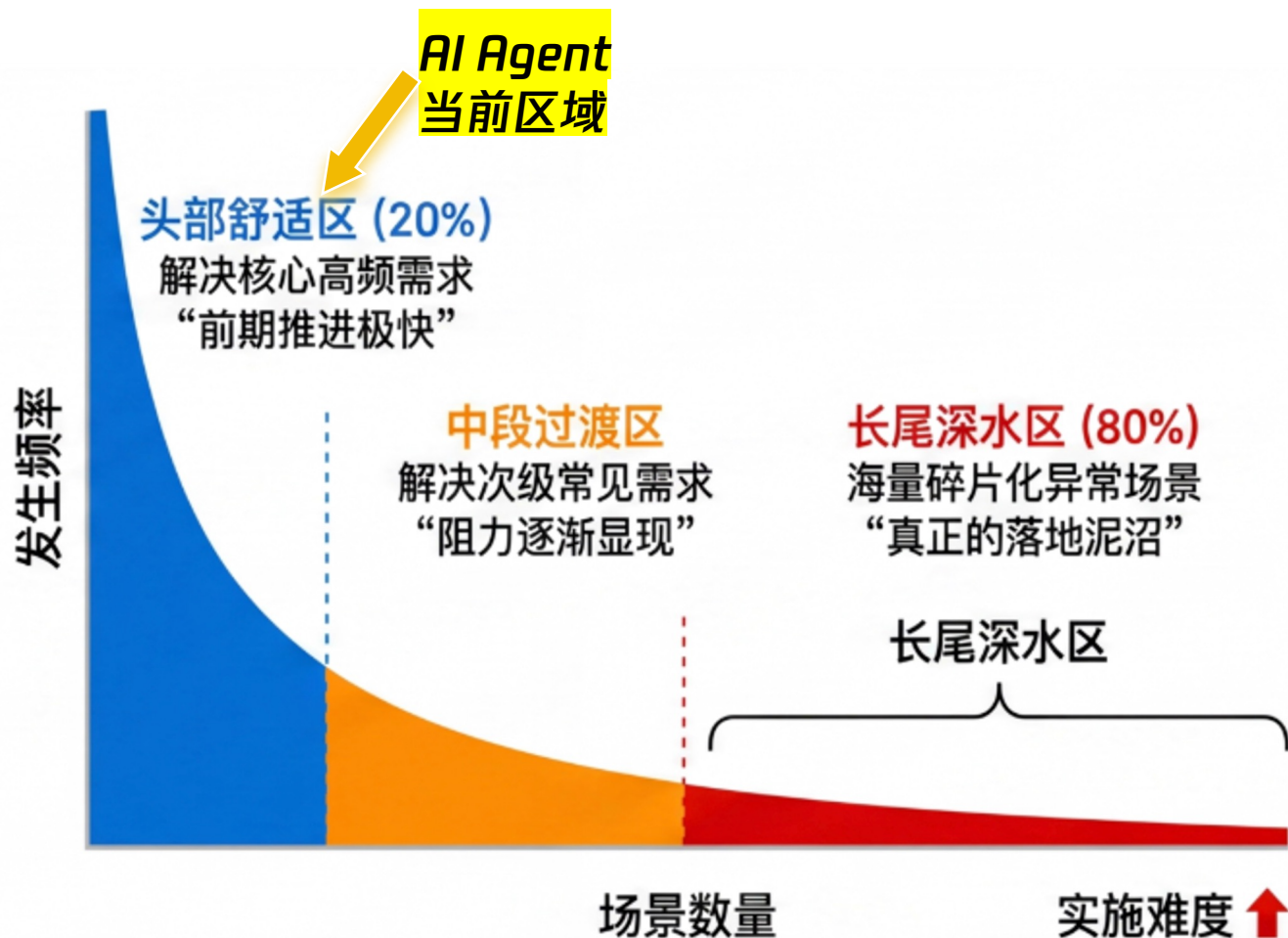
03

真实战场：碎片场景

长尾效应：真实生产环境充满变数，AI 表现可能并不如人意



长尾效应：为什么工具用户越多，研发越难做？



头部需求容易做，
真正难的是长尾场景落地

从PPT开头的数据聊起

公域**人均3.2小时以上**用量
意味着

用户深度使用

- 场景**更复杂**
- **异常更多、代价更高**
- 实现**成本更高**

更让我注意的是
有时候**用户比我们还要细心**

长尾效应中的真实需求案例

某机关单位强制国密客户端 + 国密服务器，强制严格限制发送目标

这其实才是我们打工人的日常工作

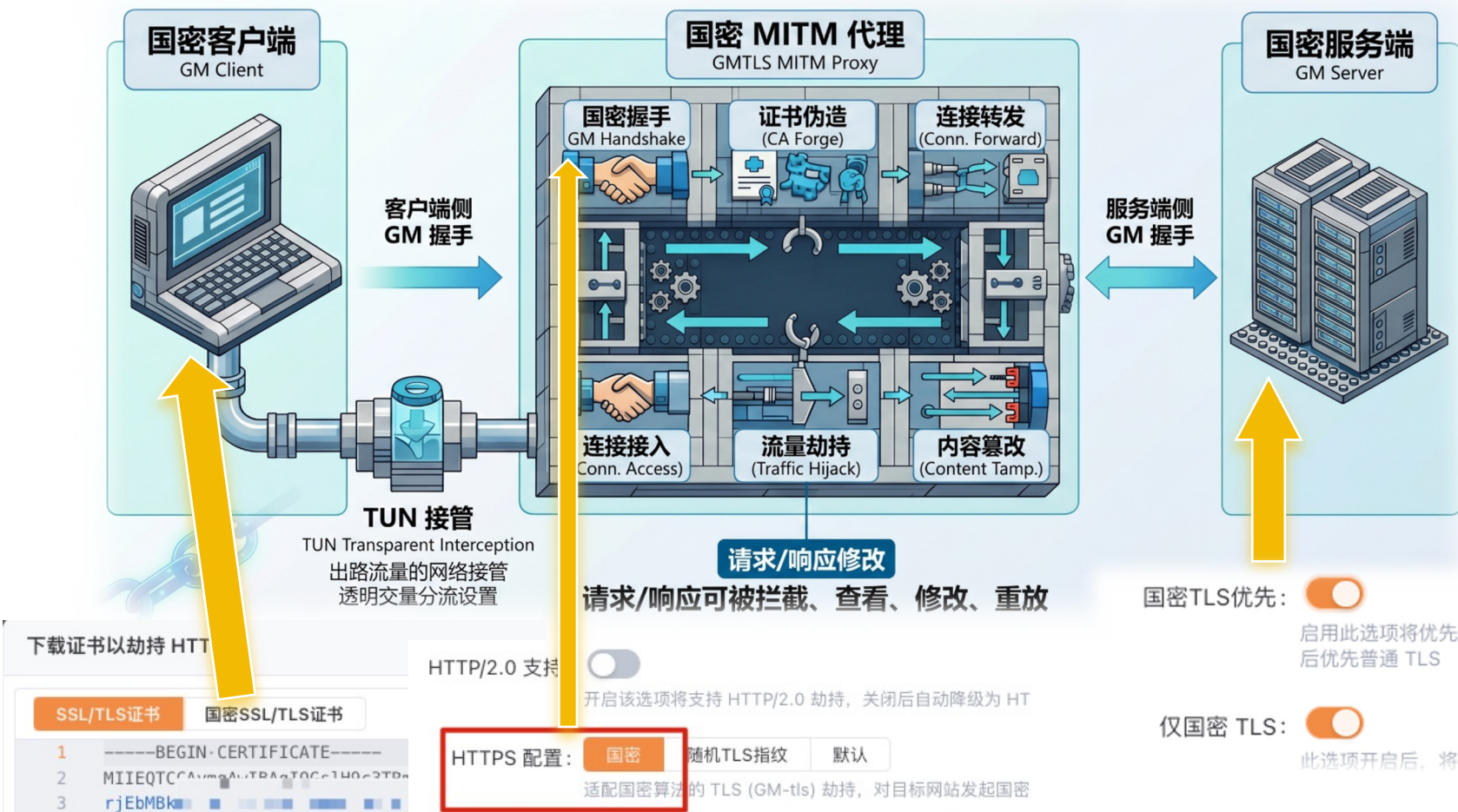
AI Agent 参与这个测试场景的前提是

完善的基础设施

这些基础设施包含了过往研发的

1. 大量被验证的边界情况；
2. 未被记录的不规范的协议实现兼容；
3. 不同系统/协议的细微差别；

AI 无法在短时间内复刻这些底层基础设施边界内容

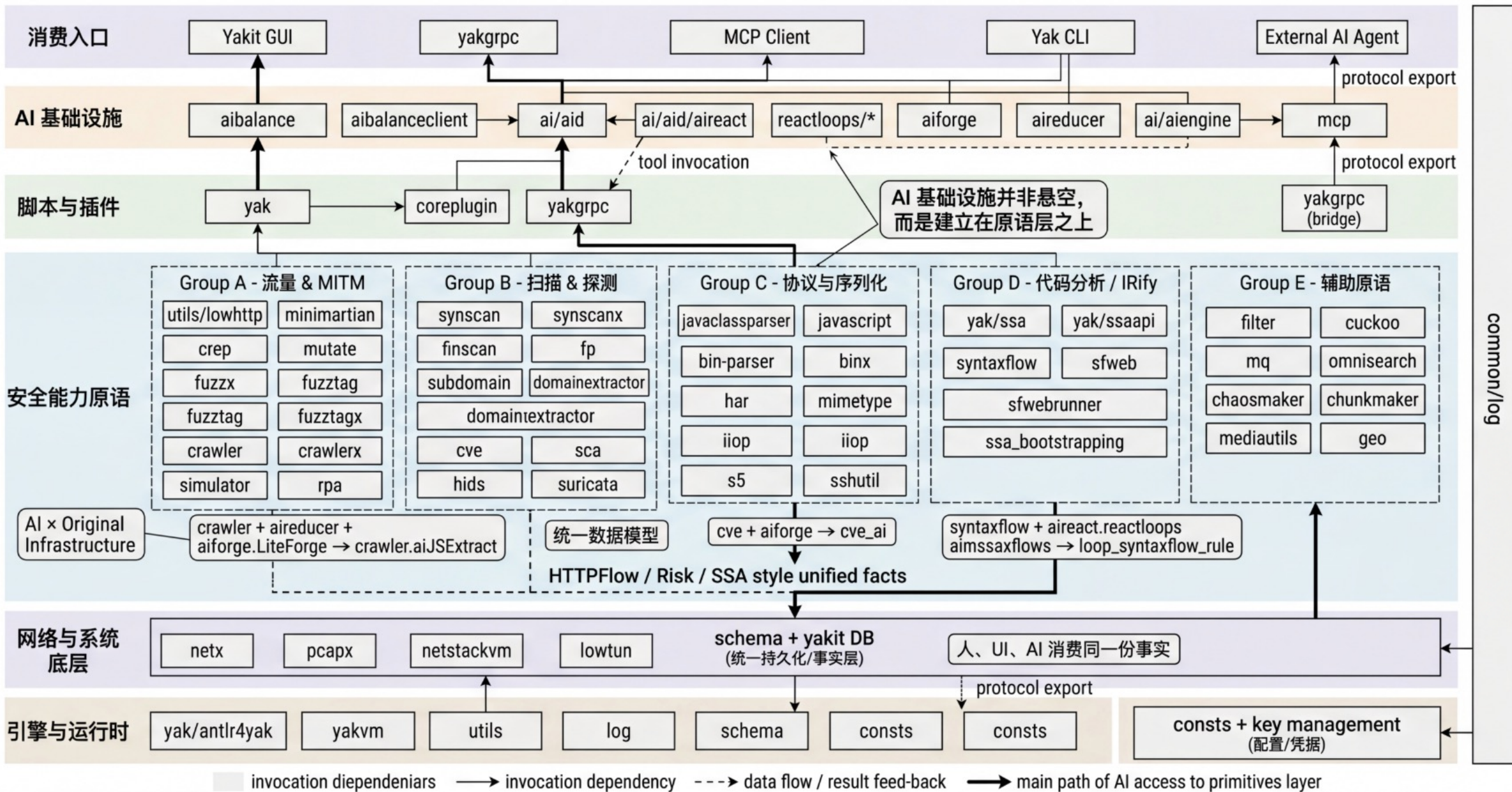


04

基础设施仍然重要

AI 依赖工具，工具使用 AI 强化场景，都依赖底层基础设施





THANKS

